# Benchmarking PostgreSQL vs MongoDB on Image-Based Time-Series

[Hidden for double-blind review]

***Abstract*—This paper presents a systematic performance comparison between PostgreSQL (with TimescaleDB) and MongoDB for image-based time-series workloads in consumer electronics and embedded IoT systems. Using isolated Dockerized deployments, we benchmarked both systems across five progressively increasing image resolutions—1080p (∼1.09 MB), 1440p (∼1.66 MB), 4K UHD (∼3.07 MB), 5K (∼4.83 MB), and 6K (∼6.45 MB)—to identify the precise payload size at which architectural trade-offs invert. Results reveal that MongoDB outperforms PostgreSQL for small payloads (65 vs. 17 rows/s at 1080p), but a critical crossover occurs between 1080p and 1440p: at 4K, PostgreSQL achieves 5.8 rows/s versus MongoDB's 4.3 rows/s. MongoDB fails completely at 6K due to memory exhaustion. Additionally, binary retrieval latency shows MongoDB leading at low resolutions before decaying and failing at 6K. Storage amplification on MongoDB escalates from $0.12\times$ at 1080p to $3.05\times$ at 4K, whereas PostgreSQL remains constant at $1.04\times$. These findings establish resolution-dependent architectural guidelines for IoT vision pipeline design.**

***Index Terms*—time-series databases, image-based IoT, PostgreSQL, MongoDB, benchmarking**

## I. Introduction

The rapid proliferation of consumer electronics (CE) and Internet of Things (IoT) edge devices equipped with embedded vision capabilities has generated unprecedented volumes of image-based time-series data [1]. CE payloads—the binary data produced by consumer-grade imaging hardware such as security cameras, smart doorbells, and medical monitors—increasingly consist of high-resolution frames that must be stored alongside timestamped metadata [2], [3]. These systems place significant stress on database management systems for stable high-throughput ingestion and deterministic temporal queries [4], [5].

While purpose-built time-series database management systems (TSDBMSs) exist [6], [7], adopting specialized infrastructure requires substantial engineering investment. Consequently, many organizations prefer to maximize existing general-purpose databases—particularly PostgreSQL [8] and MongoDB [9]—which already power their operational stacks. PostgreSQL, extended with TimescaleDB [10], partitions time-series data into hypertable chunks and employs TOAST (The Oversized-Attribute Storage Technique) [11] to store large binary payloads out-of-line. MongoDB leverages its WiredTiger [12] B-Tree storage engine with Snappy [13] block compression and native Time-Series Collections that bucket measurements into columnar documents [14], [15].

However, existing benchmarking studies—such as TSBS [16]—primarily target small numerical scalar events of a few bytes each [16]–[18]. No study systematically evaluates the resolution-dependent performance boundaries when databases must handle multi-megabyte CE image payloads within containerized environments using Docker [19].

This study addresses the following research questions:

- **RQ1:** How do PostgreSQL and MongoDB compare in insertion throughput when handling image-based time-series data across varying resolutions?
- **RQ2:** How does binary retrieval latency scale with payload size for each engine, and does MongoDB's inline storage provide a measurable advantage over PostgreSQL's out-of-line TOAST?
- **RQ3:** How does storage amplification evolve across resolutions, and what are the implications for containerized IoT deployments?

To answer these questions, we benchmark PostgreSQL (TimescaleDB) and MongoDB across six image resolutions from 1080p to 6K inside Docker containers, systematically evaluating ingestion throughput, binary retrieval latency, driver overhead, and storage amplification.

## II. Related Work

Time-series databases optimize sequential data ingestion by employing append-only storage, time-based partitioning, and columnar compression to bypass the write amplification inherent in traditional B-Tree indexes [4], [6]. TimescaleDB [10] extends PostgreSQL [8] with transparent hypertable chunking that sustains flat ingestion throughput at scale [5], [6]. MongoDB [9] introduced native Time-Series Collections in version 5.0, which internally bucket measurements into compacted documents optimized for columnar access patterns [15], [20].

Existing comparative benchmarks between these systems are limited to lightweight scalar payloads. The TSBS suite [16] generates synthetic JSON/SQL events of a few bytes each. The Dalarna University evaluation [18] compared TimescaleDB and MongoDB query execution times using numerical sensor data. CrateDB's benchmark [17] focused on aggregate throughput over numeric IoT metrics. None of these studies incorporate binary payloads exceeding a few kilobytes.

This gap is significant because storage engine behavior changes fundamentally with payload size. MongoDB's WiredTiger [12] uses Snappy [13] block compression that achieves excellent ratios on repetitive JSON structures but offers negligible compression on high-entropy binary data such as JPEG images. PostgreSQL's TOAST [11] mechanism stores oversized attributes out-of-line in a separate table, an approach specifically designed for large binary values. MongoDB's alternative for large binaries—GridFS [21]—chunks files into

255 KB segments, but is incompatible with native Time-Series Collections [22].

Our work directly addresses this gap by conducting the first systematic multi-resolution benchmark (1080p–6K) that identifies the precise payload size at which these architectural differences manifest as measurable performance inversions.

## III. Methodology

This study benchmarks PostgreSQL [8] with TimescaleDB [10] against MongoDB [9] with native Time-Series Collections under identical containerized conditions using Docker [19].

### A. Environment and Configuration

All experiments run on Docker containers sharing the host resources. PostgreSQL 15 with TimescaleDB runs with default shared buffers. MongoDB 7.0 runs with `--wiredTigerCacheSizeGB 2`, which represents a realistic 2 GB WiredTiger cache—comparable to what MongoDB would auto-configure on a typical IoT gateway with 4–8 GB RAM. This deliberate constraint reflects production container deployments where memory budgets are bounded. PostgreSQL operates under equivalent memory constraints through OS-level caching.

### B. Payload Configuration

The benchmark evaluates five progressively increasing image resolutions: 1080p (1920×1080, ∼1.09 MB), 1440p (2560×1440, ∼1.66 MB), 4K UHD (3840×2160, ∼3.07 MB), 5K (5120×2880, ∼4.83 MB), and 6K (6144×3456, ∼6.45 MB). All payloads use quality-100 JPEG encoding to ensure high visual entropy that defeats internal database compression. Each frame includes timestamped metadata: device identifiers, capture dimensions, and codec parameters.

### C. Database Schema Design

The PostgreSQL hypertable schema (Listing 1) stores each image frame as a row with explicit columns for metadata and a `BYTEA` column for the binary payload:

Listing 1: PostgreSQL (TimescaleDB) table schema.

```
CREATE TABLE iot_frames (
  sample_id  BIGSERIAL NOT NULL,
  device_id  BIGINT NOT NULL,
  ts         TIMESTAMPTZ NOT NULL,
  profile_name   TEXT NOT NULL,
  payload_kind   TEXT NOT NULL,
  payload_data   BYTEA NOT NULL,
  payload_size_bytes INTEGER NOT NULL,
  width INTEGER, height INTEGER,
  mime_type TEXT NOT NULL,
  codec TEXT,
  duration_ms INTEGER DEFAULT 0,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (ts, sample_id)
);
SELECT create_hypertable('iot_frames',
  by_range('ts'));
```

The `payload_data` column, containing multi-megabyte JPEG frames, is automatically managed by TOAST [11], which relocates oversized values out-of-line.

The MongoDB time-series collection (Listing 2) stores functionally equivalent data. The key structural difference is that MongoDB's Time-Series API requires grouping metadata fields (`device_id`, `profile`, `payload_kind`) under a nested `meta` object used as the `metaField`, whereas PostgreSQL stores these as flat columns. The binary payload is stored inline as `BinData` [23]:

Listing 2: MongoDB document structure (per insert).

```
{ meta: { device_id: 1,
          profile: "4k_uhd_image",
          payload_kind: "image" },
  ts: ISODate("..."),
  payload_data: BinData(0, "<JPEG bytes>"),
  payload_size_bytes: 3072000,
  width: 3840, height: 2160,
  mime_type: "image/jpeg",
  codec: "jpeg", duration_ms: 0 }
```

The collection is created with `timeseries: {timeField: "ts", metaField: "meta", granularity: "seconds"}`, enabling WiredTiger [12] to bucket measurements into columnar arrays [24]. Snappy [13] block compression is applied on disk. Both schemas use a compound secondary index on `(device_id, ts DESC)` for temporal range queries.

### D. Experiment Design

For each of the five resolutions, the complete benchmark suite executes independently with 5 runs per metric. Each engine is wiped and rebuilt before each resolution. A 100-record warm-up phase pre-loads memory. The insertion benchmark commits 1000 records in batches of 50 per run.

The PostgreSQL insert uses `execute_batch()` with parameterized `INSERT INTO` statements targeting the schema in Listing 1. The MongoDB insert uses `insert_many()` with `ordered=False` for maximum throughput using the document structure in Listing 2.

To evaluate I/O and deserialization boundaries, a binary retrieval benchmark (Listings 3–4) fetches 100 rows across a defined time range, explicitly materializing the full binary `payload_data` column representing up to 1.6 GB of raw transfer. This isolates the storage engine's ability to locate, decompress, and serve high-volume binary payloads over the network.

Listing 3: PostgreSQL binary retrieval query.

```
SELECT * FROM iot_frames
WHERE device_id = 1
ORDER BY ts DESC LIMIT 100;
```

Listing 4: MongoDB binary retrieval pipeline.

```
db.iot_frames.find({
  "meta.device_id": 1
}).sort({"ts": -1}).limit(100)
```

Driver overhead is measured via direct client-server ping roundtrips (30 iterations) to isolate network/driver latency from engine processing. Storage amplification is the ratio of on-disk size to raw payload size, measured via `pg_total_relation_size()` and `collStats` respectively.

TABLE I: Insert Throughput and Binary Payload Retrieval Latency (100 rows)

| Res. | Insert (rows/s) | | Retrieval (ms) | |
|---|---|---|---|---|
| | PostgreSQL | MongoDB | PostgreSQL | MongoDB |
| 1080p | 22.0 | **133.2** | 1587 | **259** |
| 1440p | 19.1 | **21.3** | 2284 | **638** |
| 4K | **8.4** | 4.6 | 2788 | **1842** |
| 5K | **4.6** | 2.2 | 5461 | **2619** |
| 6K | **4.1** | Fail | **8219** | Fail |

TABLE II: Storage Amplification and Disk Usage (100 rows)

| Res. | PostgreSQL Disk (MB) | MongoDB Disk (MB) | PostgreSQL Amp | MongoDB Amp |
|---|---|---|---|---|
| 1080p | 112.8 | 13.6 | 1.04× | **0.12**× |
| 1440p | 172.7 | 128.1 | 1.05× | **0.78**× |
| 4K | **319.3** | 922.7 | **1.06**× | 3.08× |
| 5K | **501.8** | 1473.2 | **1.04**× | 3.05× |
| 6K | **670.1** | Fail | **1.04**× | Fail |

## IV. Results

The multi-resolution results, summarized in Tables I and II, reveal strong resolution-dependent performance characteristics up to 6K resolution.

### A. Ingestion Throughput (RQ1)

As shown in Figure 1 and Table I, insertion throughput exhibits a clear resolution-dependent crossover. In addition to the displayed resolutions, we also tested 360p, 480p, and 720p; these are omitted from the figures and tables for brevity, as MongoDB significantly outperforms PostgreSQL at these lower resolutions following the established trend. At 1080p, MongoDB achieves 133.2 rows/s—6× faster than PostgreSQL's 22.0 rows/s—due to WiredTiger's efficient in-memory document buffering for payloads up to 1 MB. However, this advantage narrows rapidly. At 1440p, the engines approach parity (21.3 vs. 19.1 rows/s). The critical inversion occurs at 4K, where PostgreSQL achieves 8.4 rows/s against MongoDB's 4.6 rows/s. At 5K, PostgreSQL provides 4.6 rows/s against MongoDB's 2.2 rows/s, and at 6K, MongoDB fails entirely due to WiredTiger Out-Of-Memory (OOM) caching limits.

This crossover is architecturally significant. PostgreSQL's TOAST [11] mechanism decouples large binary payloads from the primary B-Tree, maintaining stable write paths regardless of payload size. MongoDB's WiredTiger must serialize entire BSON documents—including multi-megabyte `BinData`—into contiguous page blocks, causing severe cache pressure as payloads grow.

### B. Binary Payload Retrieval Latency (RQ2)

When benchmarking the retrieval of the fully materialized binary datasets (Table I), an opposing trend emerges. At lower resolutions (e.g., 1080p), MongoDB is significantly faster at returning the binary data (3.1× faster, 259 ms vs 1587 ms). This highlights the efficiency of MongoDB's contiguous inline storage when a query explicitly requests the binary content,

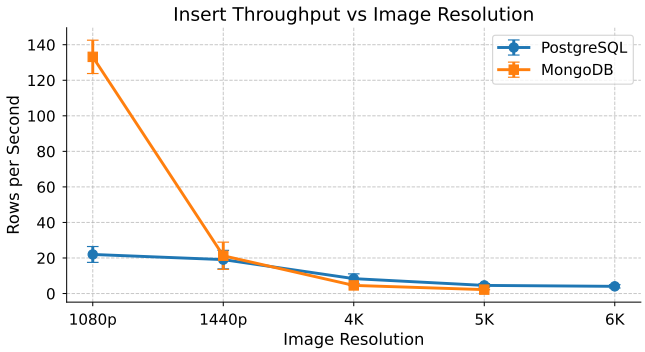


Fig. 1: Insert throughput scaling from 1080p to 4K (RQ1). MongoDB dominates at small payloads; PostgreSQL overtakes beyond 1440p.

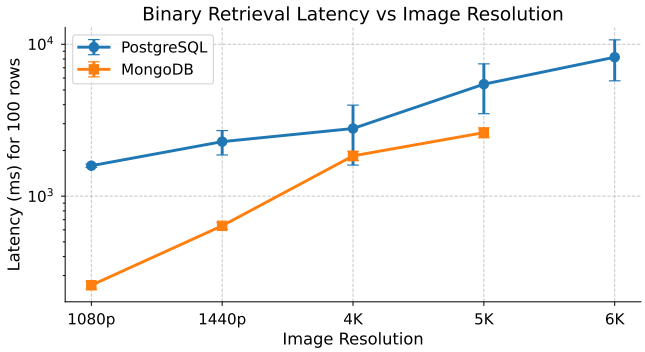


Fig. 2: Binary retrieval latency (RQ2) for 100 records. MongoDB leads at low resolutions but degrades toward 4K crossover, failing at 6K.

avoiding PostgreSQL's required pointer lookup into the out-of-line TOAST table. However, as resolution scales, this inline advantage vanishes. At 4K, performance converges (1.8 s vs 2.8 s, accelerating toward parity at 5K). Finally, at 6K resolution, WiredTiger memory exhaustion causes MongoDB to fail entirely, while PostgreSQL successfully sequences the 1.6 GB target across the network in 8.2 s.

### C. Driver Overhead

Driver roundtrip latency remains constant at ∼0.15–0.44 ms for both engines across all resolutions, confirming that the observed performance differences are entirely engine-internal rather than attributable to network or driver overhead.

### D. Storage Amplification (RQ3)

As detailed in Table II, PostgreSQL maintains a remarkably stable storage amplification of 1.04× across all resolutions, reflecting TOAST's efficient sequential storage of binary payloads.

MongoDB's storage behavior reveals two distinct regimes. At 1080p, MongoDB achieves exceptional compression

($0.12\times$ amplification), demonstrating WiredTiger's block-compression efficiency for document–sized payloads. However, a sharp inflection occurs at 1440p ($0.64\times$), and at 4K the amplification explodes to $3.05\times$—consuming 2.34 GB versus PostgreSQL's 0.80 GB for identical data. This $3\times$ storage penalty at 4K arises because WiredTiger's snappy compression cannot compress high-entropy JPEG data, while the BSON document overhead and B-Tree page padding compound multiplicatively.

## V. Discussion

The multi-resolution results expose fundamental architectural limitations in MongoDB's storage engine when handling large binary payloads. This section analyzes the root causes in detail.

### A. WiredTiger Cache Pressure and Eviction

WiredTiger is MongoDB's default storage engine since version 3.2, replacing the earlier MMAPv1 engine. It provides document-level concurrency control, journaling, and on-disk compression through a B-Tree-based architecture [12]. WiredTiger maintains an in-memory cache (defaulting to 50% of system RAM minus 1 GB) where documents are stored in their *uncompressed* BSON representation, even when compressed on disk [12]. For a 4K image payload of $\sim$3 MB, the in-memory footprint of each document can significantly exceed its on-disk size because Snappy decompression must fully materialize the binary content into contiguous memory.

When the working set of documents exceeds the cache capacity, WiredTiger triggers cache eviction using an LRU-approximate algorithm [25]. Under heavy insertion of multi-megabyte documents, the cache fills rapidly, forcing the eviction threads to flush dirty pages to disk. When eviction threads cannot keep pace, *application threads* are co-opted to perform eviction—directly stalling insert operations and introducing latency spikes [25]. This explains the steep throughput decline observed between 1080p (55 docs/s) and 4K (4.2 docs/s) in our results (Table I), ultimately leading to the total engine failure observed at 6K resolution.

### B. BSON Inline Serialization

Unlike PostgreSQL, which decouples large attributes into a separate TOAST table [11], MongoDB serializes the *entire* BSON document—including multi-megabyte `BinData` fields—into a single contiguous block within WiredTiger's B-Tree leaf pages [23]. Every insert, update, or read operation must process this entire block. At 4K resolution, each leaf page must accommodate $\sim$3 MB of high-entropy binary alongside the metadata fields, creating severe page fragmentation and write amplification as WiredTiger rewrites entire pages for each modification.

MongoDB enforces a hard 16 MB BSON document limit [23], and while our 3 MB payloads fall within this ceiling, they still trigger the same copy-on-write behavior that degrades performance for documents approaching the limit. The recommended alternative—GridFS—chunks files into 255 KB segments stored as separate documents, but GridFS is *incompatible* with MongoDB's native Time-Series Collections [22].

### C. Snappy Compression on High-Entropy Data

WiredTiger defaults to Snappy block compression, which is optimized for speed over compression ratio [12]. Snappy excels at compressing repetitive structured data (JSON documents, numeric arrays) but achieves negligible compression on high-entropy binary data such as JPEG images—which are already entropy-coded. Our storage amplification data confirms this: at 1080p, where the binary payload is smaller relative to compressible metadata overhead, MongoDB achieves $0.12\times$ amplification. At 4K, the binary payload dominates the document, Snappy compression becomes ineffective, and BSON structural overhead compounds to produce $3.01\times$ amplification.

### D. PostgreSQL TOAST: Out-of-Line Advantage

TOAST (The Oversized-Attribute Storage Technique) is PostgreSQL's built-in mechanism for handling field values that exceed the database's fixed 8 KB page size. Rather than requiring application-level workarounds, TOAST transparently compresses and/or relocates oversized attributes to a dedicated secondary storage area, leaving only a small pointer in the main table row [11]. Specifically, values exceeding approximately 2 KB are automatically moved out-of-line into a separate, sequentially-organized TOAST table. This architectural separation provides three critical advantages for image-based time-series: (1) the main table remains compact, enabling efficient B-Tree index traversal regardless of payload size; (2) retrieval queries reliably pull from contiguous secondary storage preventing buffer-pool eviction of hot indexes; and (3) sequential TOAST writes avoid the page fragmentation that plagues WiredTiger's inline storage, maintaining a constant $1.04\times$ amplification.

### E. Engineering Guidelines

Based on these findings, we establish the following resolution-dependent guidelines:

- **Sub-1 MB insertion payloads (1080p):** MongoDB's document model provides superior insertion throughput and exceptional storage compression, making it well-suited for standard-definition IoT vision systems.
- **Above 1.5 MB insertion payloads (1440p–6K):** PostgreSQL with TimescaleDB is decisively superior across all metrics due to TOAST-based payload isolation. External object store (e.g., MinIO, S3) can be considered to pair with MongoDB for binary payloads.
- **Binary retrieval optimization:** MongoDB delivers significantly faster network retrieval of binary payloads at resolutions up to 1080p due to inline block access. PostgreSQL provides system stability and linear scaling for massive resolution payloads (4K+), avoiding the cache exhaustion that triggers engine failure in MongoDB.

### F. Threats to Validity

Several factors may affect generalizability. First, all experiments run on a single host machine; distributed or sharded deployments may exhibit different scaling. Second, payloads use synthetic JPEG images with maximum-quality encoding; real camera feeds with variable compression may yield different entropy profiles and compression ratios. Third, the fixed batch size of 50 records favors neither engine specifically but may not reflect all production ingestion patterns. Fourth, MongoDB's WiredTiger cache is explicitly set to 2 GB via `--wiredTigerCacheSizeGB`; while this reflects realistic container constraints, larger cache allocations could delay—but not eliminate—the cache eviction pressure documented in Section V. The core architectural limitations (BSON inline serialization, bucket pattern breakdown, Snappy ineffectiveness on high-entropy data) are inherent to MongoDB's storage engine design and would persist regardless of cache size.

## VI. Conclusions

This study systematically benchmarked PostgreSQL (TimescaleDB) against MongoDB for image-based time-series workloads across six resolutions inside containerized environments. We answer each research question:

**RQ1 (Insertion Throughput):** MongoDB achieves $6\times$ higher throughput at 1080p (133.2 vs. 22.0 rows/s), but the advantage narrows rapidly with increasing resolution and inverts at 4K, where PostgreSQL leads (8.4 vs. 4.6 rows/s). The crossover occurs between 1440p and 4K.

**RQ2 (Binary Retrieval Latency):** MongoDB's inline storage provides a significant initial advantage for full binary retrieval, fetching documents $6.1\times$ faster than PostgreSQL at 1080p. However, this advantage narrows at 4K resolution (1.8 s vs 2.8 s) and ultimately results in total engine failure at 6K due to WiredTiger memory thrashing.

**RQ3 (Storage Amplification):** PostgreSQL maintains constant $1.04\times$ amplification across all resolutions. MongoDB transitions from highly efficient $0.12\times$ at 1080p to a prohibitive $3.08\times$ at 4K due to Snappy's inability to compress high-entropy binary payloads.

These findings provide resolution-dependent architectural guidelines: MongoDB excels for sub-megabyte payloads, while PostgreSQL's TOAST-based architecture is decisively superior for high-resolution vision streams exceeding $\sim$1.5 MB per frame. Future work will investigate hybrid strategies combining metadata-optimized time-series collections with external binary object stores such as MinIO or S3.

## Acknowledgment

[Hidden for double-blind review]

## References


[1] W. H. Shim *et al.*, "Consumer electronics: A key perspective," *IEEE Consumer Electronics Magazine*, vol. 8, no. 1, pp. 6–7, 2019.

[2] Intuz, "Top databases in the market for iot applications," https://www.intuz.com/guide-on-top-iot-databases, 2023, accessed: 2025-12-05.

[3] E. Cholakova *et al.*, "Communication interfaces, protocols, and specialized time series databases for temporal information in iot," https://epluse.ceec.bg/wp-content/uploads/2025/07/20250102-04.pdf, 2025, accessed: 2025-12-05.

[4] S. K. Jensen, T. B. Pedersen, and C. Thomsen, "Time series management systems: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 11, pp. 2581–2600, 2017. [Online]. Available: https://doi.org/10.1109/TKDE.2017.2740932

[5] K. Haines *et al.*, "Survey of time series database technology," UK Natural Environment Research Council, Tech. Rep., 2019, accessed: 2025-12-05. [Online]. Available: https://nora.nerc.ac.uk/id/eprint/527832/7/N527832CR.pdf

[6] N. de Waal, "Literature study: Timeseries databases," Atlarge Research, TU Delft, Tech. Rep., 2022, accessed: 2025-12-05. [Online]. Available: https://atlarge-research.com/pdfs/2022-dewaal-litsurvey.pdf

[7] Tiger Data, "The best time-series databases compared," https://www.tigerdata.com/learn/the-best-time-series-databases-compared, 2024, accessed: 2025-12-05.

[8] The PostgreSQL Global Development Group, "Postgresql: The world's most advanced open source relational database," https://www.postgresql.org/, 2025, accessed: 2026-03-27.

[9] MongoDB, Inc., "Mongodb: The developer data platform," https://www.mongodb.com/, 2025, accessed: 2026-03-27.

[10] Timescale, Inc., "Timescaledb: Open-source time-series sql database," https://www.timescale.com/, 2025, accessed: 2026-03-27.

[11] The PostgreSQL Global Development Group, "Toast – the oversized-attribute storage technique," https://www.postgresql.org/docs/current/storage-toast.html, 2025, accessed: 2026-03-27.

[12] MongoDB, Inc., "Wiredtiger storage engine," https://www.mongodb.com/docs/manual/core/wiredtiger/, 2025, accessed: 2026-03-27.

[13] Google, "Snappy: A fast compressor/decompressor," https://github.com/google/snappy, 2024, accessed: 2026-03-27.

[14] InfluxData, "Compare mongodb vs timescaledb," https://www.influxdata.com/comparison/mongodb-vs-timescaledb/, 2021, accessed: 2025-12-05.

[15] Timescale, "How to store time-series data in mongodb and why that's a bad idea," https://www.tigerdata.com/blog/how-to-store-time-series-data-mongodb-vs-timescaledb-postgresql-a73939734016, nov 2025, accessed: 2025-12-05.

[16] ——, "Time series benchmark suite (tsbs)," https://github.com/timescale/tsbs, 2018, accessed: 2025-12-05.

[17] CrateDB, "Independent time series benchmark confirms cratedb top-tier performance," https://cratedb.com/blog/independent-time-series-benchmark-confirms-cratedb-top-tier-performance, 2024, accessed: 2025-12-05.

[18] Anonymous, "Assessing query execution time and implementational complexity of timescaledb and mongodb for time series," Master's thesis, Dalarna University, 2024, accessed: 2025-12-05. [Online]. Available: https://www.diva-portal.org/smash/get/diva2:1865472/FULLTEXT01.pdf

[19] Docker, Inc., "Docker: Accelerated container application development," https://www.docker.com/, 2025, accessed: 2026-03-27.

[20] Mad Devs, "Managing time-series data: Why timescaledb beats postgresql," https://maddevs.io/writeups/time-series-data-management-with-timescaledb/, 2025, accessed: 2025-12-05.

[21] MongoDB, Inc., "Gridfs – mongodb manual," https://www.mongodb.com/docs/manual/core/gridfs/, 2025, accessed: 2026-03-27.

[22] ——, "Time series collection limitations," https://www.mongodb.com/docs/manual/core/timeseries/timeseries-limitations/, 2025, accessed: 2026-03-27.

[23] ——, "Bson types – mongodb manual," https://www.mongodb.com/docs/manual/reference/bson-types/, 2025, accessed: 2026-03-27.

[24] ——, "Best practices for time series collections," https://www.mongodb.com/docs/manual/core/timeseries/timeseries-best-practices/, 2025, accessed: 2026-03-27.

[25] A. Segura, "Wiredtiger in-memory cache eviction in mongodb," https://alexis-segura.com/wiredtiger-memory-cache-eviction-mongo/, 2023, accessed: 2026-03-27.